МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ЛІСОТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ

Кафедра інженерії програмного забезпечення

ЗВІТ ДО ЛАБОРАТОРНОЇ РОБОТИ №4

з навчальної дисципліни
«Алгоритми та структури даних»
На тему:
«Головні алгоритми сортування»

Виконав:
студент групи ІПЗ 11/2
Петрів О.О.
Перевірила:
доц. Павлюк У.В.

Львів – 2026

Варіант 9

9. Протабулювати задану функцію, результати табулювання занести у масив та посортувати отримані значення за зростанням методами обміну і вставки

$$y = \frac{a^x + b^{-x}}{\sin^2 bx + 1{,}24} \quad \text{де}: \ a = \begin{cases} 0{,}1 & \text{якщо} \quad x < 0{,}5 \\ 0{,}5 & \text{якщо} \quad 0{,}5 \le x \le 1 \\ 1 & \text{якщо} \quad x > 1 \end{cases};$$

b=0,15; $x_{поч}$=0,1; $x_{кін}$=2,6; крок табулювання 0,1.

Підрахувати кількість порівнянь та обмінів, що виконує кожен метод для впорядкування послідовності, та вивести отримані значення на форму

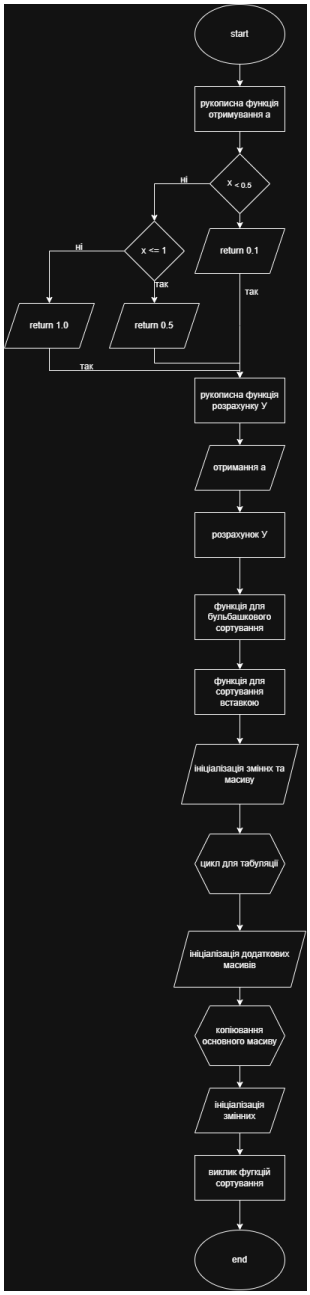
Виконання:

Код

```
#include <iostream>
#include <cmath>


double getA(double x) {
   if (x < 0.5) return 0.1;
   else if (x <= 1) return 0.5;
   else return 1.0;
}

double calcY(double x, double b) {
   double a = getA(x);
   return (pow(a, x) + pow(b, -x)) / (pow(sin(b * x), 2) + 1.24);
}

void bubbleSort(double arr[], int n, int& comp, int& swaps) {
   comp = swaps = 0;

   for (int i = 0; i < n - 1; i++) {
      for (int j = 0; j < n - i - 1; j++) {
         comp++;
         if (arr[j] > arr[j + 1]) {
            std::swap(arr[j], arr[j + 1]);
            swaps++;
         }
      }
   }
}

void insertionSort(double arr[], int n, int& comp, int& swaps) {
   comp = swaps = 0;
```

```
    for (int i = 1; i < n; i++) {
        double key = arr[i];
        int j = i - 1;

        while (j >= 0 && (++comp && arr[j] > key)) {
            arr[j + 1] = arr[j];
            j--;
            swaps++;
        }
        arr[j + 1] = key;
    }
}

int main() {
    double b = 0.15;
    double x_start = 0.1, x_end = 2.6, step = 0.1;

    int n = (x_end - x_start) / step + 1;
    double* y = new double [n];

    std::cout << "Table:\n";
    int i = 0;
    for (double x = x_start; x <= x_end + 1e-9; x += step) {
        y[i] = calcY(x, b);
        std::cout << "x=" << x << " | " << "\ty=" << y[i] << std::endl;
        i++;
    }

    double*   y1 = new double [n];
    double* y2 = new double[n];
    for (int i = 0; i < n; i++) {
        y1[i] = y[i];
        y2[i] = y[i];
    }

    int comp1, swap1, comp2, swap2;

    bubbleSort(y1, n, comp1, swap1);
    std::cout << "\nBubble sorted:\n";
    for (int i = 0; i < n; i++) std::cout << y1[i] << " ";
    std::cout << "\nComparisons: " << comp1 << " Swaps: " << swap1 << std::endl;

    insertionSort(y2, n, comp2, swap2);
    std::cout << "\nInsertion sorted:\n";
    for (int i = 0; i < n; i++) std::cout << y2[i] << " ";
    std::cout << "\nComparisons: " << comp2 << " Swaps: " << swap2 << std::endl;

    return 0;
}
```

Блок схема:



Висновок:

Сортування бульбашкою та вставками - це прості алгоритми впорядкування даних. Вони легкі у реалізації та підходять для навчання. Однак при великих масивах працюють повільно, тому на практиці часто використовують більш ефективні методи сортування.